

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS — 1ère année — Département de français

Durée : 1h30

Année universitaire : 2025-2026

Objectifs pédagogiques

À l'issue de ce TP, l'étudiant est capable de :

- Utiliser les structures conditionnelles `if/elif/else`
- Écrire des conditions complexes avec `and`, `or`, `not`
- Gérer les erreurs de saisie avec `try/except`
- Valider des données entrées par l'utilisateur
- Appliquer les conditions à des problèmes concrets (classification, validation)

Plan du TP

Phase	Contenu	Durée
Accueil	Rappel sur les conditions	10 min
Exercice 1	Vérification de parité	15 min
Exercice 2	Mention selon note	15 min
Exercice 3	Catégorie d'âge	15 min
Exercice 4	Convertisseur de notes (lettres → chiffres)	20 min
Exercice 5	Validation de mot de passe	15 min

Prérequis

- TP 01 bouclé (variables, `print()`, `input()`, conversions de types)
- Python 3 installé et fonctionnel

Exercice 1 — Vérification de parité (15 min)

Énoncé : Créer un script `parite.py` qui :

1. Demande un nombre entier à l'utilisateur
2. Affiche si le nombre est pair ou impair
3. Affiche également si le nombre est divisible par 3 et par 5

Exemple attendu :

```
Entrez un nombre : 15
15 est impair
15 est divisible par 3 et par 5
```

Corrigé :

```
try:
    n = int(input("Entrez un nombre : "))
    if n % 2 == 0:
        print(f"{n} est pair")
    else:
        print(f"{n} est impair")
    if n % 3 == 0 and n % 5 == 0:
        print(f"{n} est divisible par 3 et par 5")
    elif n % 3 == 0:
        print(f"{n} est divisible par 3")
    elif n % 5 == 0:
        print(f"{n} est divisible par 5")
except ValueError:
    print("Veuillez entrer un nombre entier valide.")
```

Exercice 2 — Mention selon note (15 min)

Énoncé : Créer un script `mention.py` qui demande une moyenne au bac (0-20) et affiche la mention :

- ≥ 16 : « Très bien »
- ≥ 14 : « Bien »
- ≥ 12 : « Assez bien »
- ≥ 10 : « Passable »
- < 10 : « Non admis »

Gérer le cas où la note n'est pas entre 0 et 20.

Exemple attendu :

```
Moyenne au bac (0-20) : 15
Mention Bien
```

Corrigé :

```
try:
    moyenne = float(input("Moyenne au bac (0-20) : "))
    if moyenne < 0 or moyenne > 20:
        print("La moyenne doit être entre 0 et 20.")
    elif moyenne >= 16:
        print("Mention Très bien")
    elif moyenne >= 14:
        print("Mention Bien")
    elif moyenne >= 12:
        print("Mention Assez bien")
    elif moyenne >= 10:
        print("Passable")
    else:
        print("Non admis")
except ValueError:
    print("Erreur : veuillez entrer un nombre valide.")
```

Exercice 3 — Catégorie d'âge (15 min)

Énoncé : Créer un script `categorie.py` qui demande l'âge d'une personne et affiche la tranche d'âge :

- 0-2 ans : Bébé
- 3-12 ans : Enfant
- 13-17 ans : Adolescent
- 18-59 ans : Adulte
- 60+ ans : Senior
- < 0 : Âge invalide

Exemple attendu :

Âge : 25
Catégorie : Adulte

Corrigé :

```
try:
    age = int(input("Âge : "))
    if age < 0:
        print("Âge invalide.")
    elif age <= 2:
        print("Catégorie : Bébé")
    elif age <= 12:
        print("Catégorie : Enfant")
    elif age <= 17:
        print("Catégorie : Adolescent")
    elif age <= 59:
        print("Catégorie : Adulte")
    else:
        print("Catégorie : Senior")
except ValueError:
    print("Veuillez entrer un âge valide.")
```

Exercice 4 — Convertisseur de notes (lettres → chiffres) (20 min)

Énoncé : Créer un script `notes_lettres.py` qui convertit une note littérale en note chiffrée sur 20 selon le tableau :

- A+ → 19, A → 17, A- → 16
- B+ → 15, B → 14, B- → 13
- C+ → 12, C → 11, C- → 10
- D+ → 9, D → 8, D- → 7
- E → 5, F → 2

L'utilisateur saisit une lettre (avec ou sans + ou -) et obtient la note sur 20.

Exemple attendu :

Note littérale (A+, B, C-, etc.) : B+
Note chiffrée : 15/20

Corrigé :

```
note = input("Note littérale (A+, B, C-, etc.) : ").upper()

conversion = {
    "A+": 19, "A": 17, "A-": 16,
    "B+": 15, "B": 14, "B-": 13,
    "C+": 12, "C": 11, "C-": 10,
```

```

"D+": 9, "D": 8, "D-": 7,
"E": 5, "F": 2
}

if note in conversion:
    print(f"Note chiffrée : {conversion[note]}/20")
else:
    print("Note littérale invalide.")

```

Exercice 5 — Validation de mot de passe (15 min)

Énoncé : Créer un script `mdp_valide.py` qui valide un mot de passe selon ces critères :

- Au moins 8 caractères
- Contient au moins une majuscule
- Contient au moins un chiffre
- Contient au moins un symbole parmi `!@#$$%`

Indice : Utiliser les méthodes `isupper()`, `isdigit()`, et `any()`.

Exemple attendu :

```

Mot de passe : Python@2025
Mot de passe valide.

```

Corrigé :

```

mdp = input("Mot de passe : ")

longueur_ok = len(mdp) >= 8
majuscule_ok = any(c.isupper() for c in mdp)
chiffre_ok = any(c.isdigit() for c in mdp)
symbole_ok = any(c in "!@#$$%" for c in mdp)

if longueur_ok and majuscule_ok and chiffre_ok and symbole_ok:
    print("Mot de passe valide.")
else:
    print("Mot de passe invalide.")
    if not longueur_ok:
        print(" - Au moins 8 caractères")
    if not majuscule_ok:
        print(" - Au moins une majuscule")
    if not chiffre_ok:
        print(" - Au moins un chiffre")
    if not symbole_ok:
        print(" - Au moins un symbole (!@#$$%)")

```

Logique des conditions imbriquées (convertisseur de notes) :

```

flowchart TD
    A[Début] --> B[Saisir note littérale]
    B --> C[Normaliser en majuscules]
    C --> D{Note dans le dictionnaire ?}
    D -->|Oui| E[Récupérer valeur chiffrée]
    E --> F[Afficher note/20]
    D -->|Non| G[Afficher erreur]
    F --> H[Fin]
    G --> H

```

Structure `if/elif/else` générale :

```

flowchart TD
  A[Condition 1] -->|Vrai| B[Bloc 1]
  A -->|Faux| C[Condition 2]
  C -->|Vrai| D[Bloc 2]
  C -->|Faux| E[Condition 3]
  E -->|Vrai| F[Bloc 3]
  E -->|Faux| G[else: Bloc final]

```

QCM d'auto-évaluation

- Que retourne l'expression `not (10 > 5)` ?
 - a) True
 - b) False ✓
 - c) None
- Quelle est la sortie de `print("A" if 3 > 5 else "B")` ?
 - a) A
 - b) B ✓
 - c) True
- Quel est le bon opérateur pour tester l'égalité en Python ?
 - a) =
 - b) == ✓
 - c) !=
- Dans un `elif`, combien de `else` peut-on avoir ?
 - a) 0 ou 1 ✓
 - b) Autant qu'on veut
 - c) Au moins 1
- Que fait `try/except` ?
 - a) Répète un bloc de code
 - b) Capture et gère les erreurs ✓
 - c) Définit une fonction

Grille d'évaluation TP 02

Critère	Poids	Non acquis (0)	Partiel (0.5)	Acquis (1)
Syntaxe conditionnelle	20 %	if/elif/else incorrects	Ordre logique partiel	Structure correcte
Opérateurs logiques (and/or/not)	20 %	N'utilise pas d'opérateurs	Utilisation partielle	Utilisation correcte
Gestion d'erreurs (try/except)	20 %	Aucune gestion	Gestion partielle	Gestion complète
Dictionnaire et conversion	20 %	Pas de dictionnaire	Utilisation incorrecte	Utilisation correcte
Validation des entrées	20 %	Aucune validation	Validation minimale	Validation robuste

Déroulement séance — 1h30 (90 min)

Phase	Durée	Total cumulé
Accueil et rappel conditions	10 min	10 min
Exercice 1 — Parité	15 min	25 min
Exercice 2 — Mention selon note	15 min	40 min
Exercice 3 — Catégorie d'âge	15 min	55 min
Exercice 4 — Convertisseur notes	20 min	75 min
Exercice 5 — Validation mot de passe	15 min	90 min

Note pédagogique ENS

Ce TP introduit les structures conditionnelles, pierre angulaire de la logique algorithmique. Pour les futurs enseignants, chaque exercice est choisi pour son potentiel de réutilisation en classe : la vérification de parité (ex. 1) est un classique de l'algorithmique au collège ; le système de mentions (ex. 2) ancre l'informatique dans le quotidien scolaire ; la catégorie d'âge (ex. 3) permet de travailler les intervalles et bornes ; le convertisseur de notes lettres → chiffres (ex. 4) est une application directe du dictionnaire Python et fait le lien avec le système d'évaluation anglo-saxon, utile en cours d'anglais ou de DNL. L'exercice de validation de mot de passe (ex. 5) sensibilise à la sécurité informatique dès le collège et peut être enrichi en ajoutant des critères de robustesse (longueur minimale, diversité des caractères). L'enseignant peut proposer aux étudiants de créer une grille d'évaluation automatique qui attribue un score de sécurité au mot de passe.

Prolongement possible : Ajouter un compteur de force du mot de passe (faible / moyen / fort) en fonction du nombre de critères validés, et proposer un générateur de mot de passe sécurisé.